

Plataforma TicketU — Microservicio de Entradas e Inventario

Especificación Técnica e Ingeniería de Base de Datos (Nivel Industrial)

Asignatura: Taller de Integración Tecnológica (TITEC 2026-2)

Equipo: Grupo 3 — Microservicio Entradas / Inventario

Motor de Persistencia: MongoDB 7.0 (Arquitectura Contenerizada Local con Validación Estricta \$jsonSchema)

Estado del Documento: Versión 1.0 — Producción / Entrega Evaluación 1

1. Resumen Ejecutivo y Alcance Arquitectónico

El microservicio Entradas / Inventario constituye el núcleo transaccional del ecosistema TicketU, operando bajo el patrón arquitectónico Database per Service (base de datos independiente por microservicio). Su responsabilidad principal es actuar como la única fuente de verdad (Single Source of Truth) respecto al aforo de eventos, la retención temporal de cupos durante procesos de compra concurrentes, la consolidación financiera asíncrona y la emisión inmutable de entradas digitales con códigos QR.

- **Soporte Completo a Historias de Usuario (Criterio BD1):** Cubrir el 100% de los requerimientos funcionales de consulta de stock en tiempo real, validación de límites, bloqueo temporal con expiración automática (TTL), diferenciación entre flujo gratuito y pagado, y generación de identificadores QR.
 - **Aislamiento Estricto de Dominio (Criterio BD2):** Almacenar exclusivamente datos generados por el dominio de Entradas/Inventario, referenciando entidades externas únicamente mediante sus identificadores (`id_evento`, `id_usuario`, `id_promocion`, `id_pago`).
 - **Estandarización y Gobierno de Datos (Criterio BD3):** Aplicar de forma rigurosa la convención `snake_case` en español y en singular sobre la totalidad de las colecciones, atributos, índices y restricciones.
 - **Integridad Relacional y Topología Acíclica (Criterio BD4):** Imponer reglas de validación \$jsonSchema que aseguren claves primarias, claves foráneas, tipado BSON estricto, nulabilidad explícita y una jerarquía de Grafo Dirigido Acíclico (DAG) libre de ciclos.
-

2. Arquitectura de Integración y Flujo Transaccional de Datos

El diseño responde a una arquitectura híbrida con comunicaciones síncronas (HTTP REST) y asíncronas (RabbitMQ):

- **Fase 0 — Sincronización de Eventos:** El microservicio escucha eventos de creación/edición desde el Panel Organizador para reflejar el aforo y estado del evento.
 - **Fase 1 — Solicitud de Compra y Reserva:** Al iniciar una compra, se valida el usuario, la disponibilidad y se genera un bloqueo temporal en `reserva_entrada` con TTL de 15 minutos.
 - **Fase 2 — Evaluación de Descuentos e Inicio de Pago:** Se consulta al MS de Promociones y se inicia el cobro contra el MS de Pagos.
 - **Fase 3 — Consolidación y Distribución:** Al aprobarse el pago, se anula la fecha de expiración, se emite el registro inmutable en la colección `entrada` y se envían notificaciones a Catálogo, Check-in y Notificaciones de forma asíncrona.
 - **Fase 4 — Agotamiento de Stock:** Si el stock llega a cero, se notifica inmediatamente al Panel Organizador (estado AGOTADO).
-

3. Estándares de Nomenclatura y Gobierno de Datos

Se establecieron las siguientes normas obligatorias: - **Escritura:** Uso exclusivo de `snake_case` sin caracteres especiales (ej. `cantidad_entrada_reservada`). - **Idioma y Número:** Español en singular (ej.

inventario_evento). - **Claves Primarias (PK):** Prefijo `id_` con índice UNIQUE (ej. `id_evento`). - **Claves Foráneas (FK y FK Ext):** Prefijos `id_` relacionando colecciones internas y externas sin duplicar datos. - **Atributos Temporales:** Prefijo `fecha_` en formato BSON Date (ej. `fecha_creacion`). - **Estados/Tipos:** Enumeraciones cerradas validadas por `$jsonSchema` en UPPER_CASE (ej. "PAGADA").

4. Aislamiento de Dominio y Matriz de Dependencias

Cero datos de negocio de otros squads: no existen nombres, correos ni datos transaccionales foráneos en las tablas, únicamente se conservan las referencias externas. La dupla plana de `id_usuario` y `rol_usuario` permite identificar si un usuario actúa como organizador o cliente, interactuando con Auth de manera asilada.

- En `inventario_evento`, `rol_usuario` exige el valor "ORGANIZADOR".
- En `reserva_entrada` y `entrada`, `rol_usuario` exige el valor "CLIENTE".

5. Topología Relacional en Cascada Lineal sin Ciclos

La arquitectura base carece de triángulos de dependencias transitivas gracias a un flujo unidireccional:

`inventario_evento -> reserva_entrada -> entrada`

Una `reserva_entrada` depende de un `inventario_evento`. A su vez, una `entrada` depende única y exclusivamente de la `reserva_entrada`. Esto impide dependencias múltiples que ocasionen ciclos (se descartó relacionar `entrada` con `inventario_evento` directamente).

6. Diccionario de Datos Exhaustivo

6.1. Colección 1: `inventario_evento`

Atributo	Tipo BSON	Clave	Nullable	Regla \$jsonSchema	Descripción Funcional
<code>_id</code>	<code>objectId</code>	Interno	NOT NULL	Automático	ID interno de Mongo.
<code>id_evento</code>	<code>string</code>	[PK]	NOT NULL	UNIQUE	Identificador de negocio del evento.
<code>id_usuario</code>	<code>string</code>	[FK Ext]	NOT NULL	String	Organizador del evento.
<code>rol_usuario</code>	<code>string</code>	—	NOT NULL	enum: ["ORGANIZADOR"]	Rol del usuario creador.
<code>tipo_entrada</code>	<code>string</code>	—	NOT NULL	enum: ["PAGADA", "GRATUITA"]	Tipo de evento.
<code>precio_unitario</code>	<code>int</code>	—	NOT NULL	minimum: 0	Precio base del evento.
<code>stock_inicial</code>	<code>int</code>	—	NOT NULL	minimum: 0	Aforo total original.
<code>stock_actual</code>	<code>int</code>	—	NOT NULL	minimum: 0	Cupos disponibles reales.

Atributo	Tipo BSON	Clave	Nullable	Regla \$jsonSchema	Descripción Funcional
cantidad_entrada_int	int	—	NOT NULL	minimum: 0	Total temporalmente retenido.
cantidad_entrada_int	int	—	NOT NULL	minimum: 0	Total efectivamente pagado.
maximo_ticket	int	—	NOT NULL	minimum: 1	Máximo por transacción.
estado_gestion	string	—	NOT NULL	enum	Estado (PUBLICADO, AGO-TADO...).
fecha_actualizacion	date	—	NOT NULL	Date	Última modificación.

6.2. Colección 2: reserva_entrada

Atributo	Tipo BSON	Clave	Nullable	Regla \$jsonSchema	Descripción Funcional
_id	objectId	Interno	NOT NULL	Automático	ID interno de Mongo.
id_reserva	string	[PK]	NOT NULL	UNIQUE	Identificador de retención temporal.
id_evento	string	[FK]	NOT NULL	String	Relación al inventario.
id_usuario	string	[FK Ext]	NOT NULL	String	Cliente solicitante.
rol_usuario	string	—	NOT NULL	enum: ["CLIENTE"]	Rol de comprador.
cantidad_entrada_int	int	—	NOT NULL	minimum: 1	Cantidad reservada.
id_promocion	string	[FK Ext]	NULLABLE	["string", "null"]	Referencia de promoción.
porcentaje_descuento	int	—	NOT NULL	min:0, max:100	Descuento aplicado.
monto_total	int	—	NOT NULL	minimum: 0	Total a cobrar en Pasarela.
id_pago	string	[FK Ext]	NULLABLE	["string", "null"]	Transacción externa de pago.
estado_reserva	string	—	NOT NULL	enum	Estado (RESERVADO, CONSOLIDADO...).
fecha_creacion	date	—	NOT NULL	Date	Inicio de la retención.
fecha_expiracion	date	Índice TTL	NULLABLE	["date", "null"]	Tiempo límite para pagar.

inventario_evento		
objectId	_id	NOT NULL
string	id_evento	[PK] NOT NULL
string	id_usuario	[FK Ext] NOT NULL
string	rol_usuario	NOT NULL
string	tipo_entrada	NOT NULL
int	precio_unitario	NOT NULL
int	stock_inicial	NOT NULL
int	stock_actual	NOT NULL
int	cantidad_entrada_reservada	NOT NULL
int	cantidad_entrada_comprada	NOT NULL
int	maximo_ticket	NOT NULL
string	estado_gestion	NOT NULL
date	fecha_actualizacion	NOT NULL



 tiene (via id_evento)

reserva_entrada		
objectId	_id	NOT NULL
string	id_reserva	[PK] NOT NULL
string	id_evento	[FK] NOT NULL
string	id_usuario	[FK Ext] NOT NULL
string	rol_usuario	NOT NULL
int	cantidad_entrada	NOT NULL
string	id_promocion	[FK Ext] NULLABLE
int	porcentaje_descuento	NOT NULL
int	monto_total	NOT NULL
string	id_pago	[FK Ext] NULLABLE
string	estado_reserva	NOT NULL
date	fecha_creacion	NOT NULL
date	fecha_expiracion	NULLABLE



 genera (via id_reserva)

entrada		
objectId	_id	NOT NULL
string	id_entrada	[PK] NOT NULL
string	id_reserva	[FK] NOT NULL
string	id_usuario	[FK Ext] NOT NULL
string	rol_usuario	NOT NULL
string	tipo_acceso	NOT NULL
int	precio_final_pagado	NOT NULL
string	estado_entrada	NOT NULL
string	nombre_archivo_qr	NOT NULL
string	qr_data	NOT NULL
date	fecha_emision	NOT NULL

Figure 1: Diagrama Relacional

6.3. Colección 3: entrada

Atributo	Tipo BSON	Clave	Nullable	Regla \$jsonSchema	Descripción Funcional
_id	objectId	Interno	NOT NULL	Automático	ID interno de Mongo.
id_entrada	string	[PK]	NOT NULL	UNIQUE	Ticket generado inmutable.
id_reserva	string	[FK]	NOT NULL	String	Reserva de origen.
id_usuario	string	[FK Ext]	NOT NULL	String	Titular de la entrada.
rol_usuario	string	—	NOT NULL	enum: ["CLIENTE"]	Rol del titular.
tipo_acceso	string	—	NOT NULL	enum: ["GENERAL"]	Modalidad de acceso.
precio_final_pagado	number	—	NOT NULL	minimum: 0	Cobro real liquidado.
estado_entrada	string	—	NOT NULL	enum: ["EMITIDA", "ANULADA"]	Estado operativo del ticket.
nombre_archivo_qr	string	—	NOT NULL	String	Nombre del archivo de imagen.
qr_data	string	—	NOT NULL	String	URL del recurso QR en bucket.
fecha_emision	date	—	NOT NULL	Date	Instante de generación del ticket.

7. Catálogo de Índices y Expiración Automática TTL

La colección `reserva_entrada` incluye un índice inteligente **TTL (Time-To-Live)** sobre `fecha_expiracion`. Esto le otorga a MongoDB la responsabilidad de purgar automáticamente aquellas reservas (bloqueos de 15 minutos) que no logren ser liquidadas con el servicio de Pagos, garantizando el reintegro de stock automático.

Existen también índices únicos (UNIQUE) para todas las PK lógicas (`id_evento`, `id_reserva`, `id_entrada`) y compuestos para búsquedas históricas rápidas (`id_usuario + id_evento`).

8. Matriz de Trazabilidad (Historias de Usuario y Contratos)

- **HU1 (Visualizar Tipo/Stock):** Cumplido en `inventario_evento.stock_actual` y `tipo_entrada`.
- **HU2 (Validación Límites):** Cumplido en `maximo_ticket` y `cantidad_entrada` con límites numéricos.
- **HU3 (Reserva TTL):** Cumplido con `fecha_expiracion` y TTL Index.
- **HU4 (Precio Inmutable):** Cumplido con `precio_final_pagado` estricto en la colección `entrada`.
- **HU5 (Registro de Archivo QR):** Cumplido almacenando URL directa en `qr_data`.
- **HU6 (Alerta Stock 0):** Validado cuando `stock_actual` de `inventario_evento` decrece hasta cero.
- **HU7 (Límite histórico):** Soportado nativamente por índices de búsqueda compuestos.
- **HU8 (Consolidación Pagos):** Relación asíncrona perfecta usando `id_pago`.

9. Manual de Operaciones (Runbook), Docker Compose y Panel Visual

Despliegue Local:

`docker compose up -d`

- **Conexión de Backend:** `mongodb://localhost:27017/ticketu_entradas_db`
- **Panel Gráfico Administrativo (Mongo Express):** Disponible en `http://localhost:8081`. El panel opera bajo la misma red docker y permite gestión 100% visual de toda la topología descrita, facilitando la auditoría e inserción de datos semilla en las pruebas cruzadas del equipo.